telecom

MDPI

Article

Adaptive Slotframe Allocation with QoS and Energy Optimization in 6TiSCH for Industrial IoT Applications

Nilam Pradhan, Bharat S. Chaudhari * and Prasad D. Khandekar

Department of Electrical and Electronics Engineering, Dr. Vishwanath Karad MIT World Peace University, Pune 411038, India; nilam.pradhan@mitwpu.edu.in (N.P.); prasad.khandekar@mitwpu.edu.in (P.D.K.)
* Correspondence: bharat.chaudhari@mitwpu.edu.in

Abstract: Industry 4.0 has transformed manufacturing and automation by integrating cyber–physical systems with the Industrial Internet of Things (IIoT) for real-time monitoring, intelligent control, and data-driven decision making. The IIoT increasingly relies on IEEE 802.15.4e Time-Slotted Channel Hopping (TSCH) to achieve reliable, low-latency, and energy-efficient industrial communications. The 6TiSCH protocol stack integrates scheduling and routing to optimize transmissions for resource-constrained devices, enhancing Quality of Service (QoS) in IIoT deployments. This paper proposes an innovative adaptive and cross-layer slotframe allocation technique for 6TiSCH networks, dynamically scheduling cells based on node hop distance, queue backlog, predicted traffic load, and link quality metrics. By dynamically adapting to these parameters, the proposed method significantly improves key QoS metrics, including end-to-end latency, packet delivery ratio, and network lifetime. The mechanism integrates real-time queue backlog monitoring, link performance analysis, and energy harvesting awareness to optimize cell scheduling decisions proactively. The results demonstrate that the proposed strategy reduces end-to-end latency by up to 32%, enhances PDR by up to 27%, and extends network lifetime by up to 10% compared to state-of-the-art adaptive scheduling solutions.

Keywords: 6TiSCH; autonomous; adaptive; scheduling technique; IIoT; latency; PDR

check for updates

Received: 26 April 2025
Revised: 29 May 2025
Accepted: 6 June 2025
Published: 10 June 2025

Citation: Pradhan, N.; Chaudhari, B.S.; Khandekar, P.D. Adaptive Slotframe Allocation with QoS and Energy Optimization in 6TiSCH for Industrial IoT Applications. *Telecom* **2025**, *6*, 41. https://doi.org/10.3390/telecom6020041

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

In the Industrial Internet of Things (IIoT) network, hundreds of ultra-low power, resource-constrained smart, and self-configured sensor nodes interact with each other over a short distance [1]. These interoperable nodes are equipped for manufacturing, processing, and monitoring tasks in the industrial automation sectors, such as the automobile industry, oil industry, waste management, forestry fire prevention, and others. The IEEE 802.15.4e Time Slotted Channel Hopping (TSCH) Medium Access Control (MAC), an enhancement over basic IEEE 802.15.4, was the key technology for low-power multi-hop Wireless Sensor Network (WSN) applications [2]. However, the harsh environmental conditions in the industry pose significant challenges in wireless communication, demanding robust and predictable performance. Latency, battery lifetime [3], Packet Delivery Ratio (PDR), throughput, and reliability serve as significant performance indicators for ensuring seamless communication under high traffic loads and external noise. Productivity and safety are the main objectives in IIoT networks that need strict resource provisions, as compared to consumer IoT [4]. To address these challenges in IIoT, several industrial communication protocols are being proposed. These protocols enable time synchronization by segmenting the slotframe into time slots and reducing external interference through channel hopping.

They have demonstrated the ability to maintain Quality of Service (QoS) in networks while ensuring ultra-low power consumption.

Devices within a TSCH network rely on a shared communication schedule, typically represented as a matrix where each cell corresponds to a unique coordinate defined by a specific time offset and channel offset. The matrix row represents channel offsets, and the column represents time offsets as shown in Figure 1. It uses the channel hopping mechanism to reduce the interference and multipath fading effect. The timeslot duration is made to be sufficiently long to allow the transmission of a packet to a neighboring node and the reception of its acknowledgment. Figure 1 also shows the basic slotframe structure for Enhanced Beacon (EB) and application data.

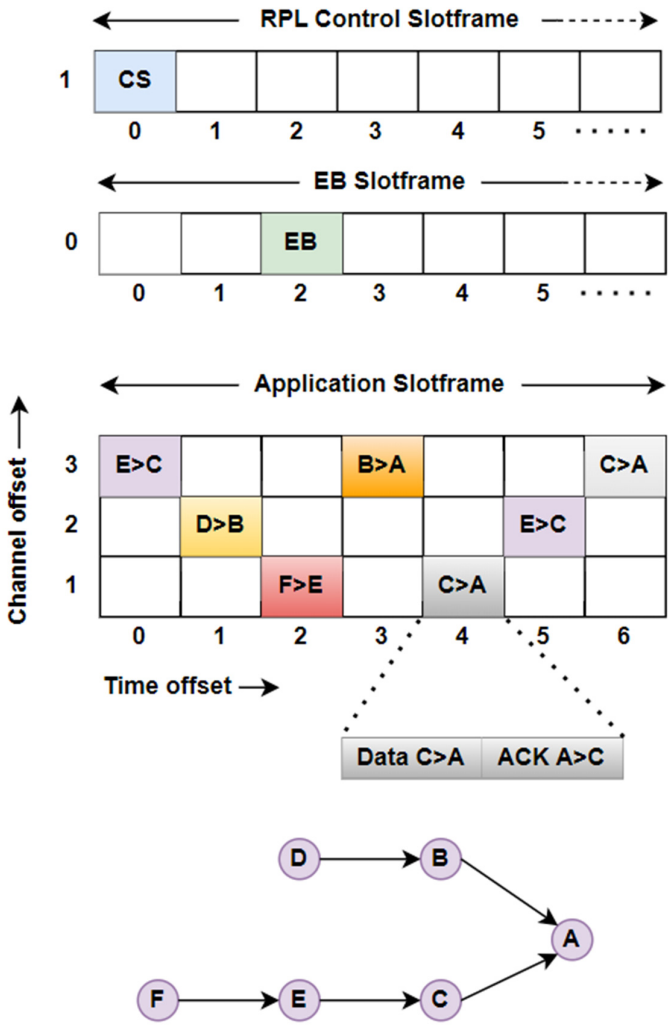

Figure 1. TSCH schedule and slotframe structure for RPL control message, enhanced beacon transmission, and application data illustrating time and channel offsets.

The operational channel frequency $f_{channel}$ for communication is determined from the channel offset as

$$f_{channel} = (ASN + channelOffset) \% c_{nu} \quad (1)$$

where ASN is the Absolute Slot Number, and c_{nu} is a number of channel offsets used. A limitation of the IEEE 802.15.4e TSCH standard is that it does not comprise rules for creating and managing the schedule in WSNs. Various standardization groups are developing innovative solutions to integrate industrial automation with Internet services. They standardize an architecture for the IIoT, allowing every device in the industry to

interact globally. This paper introduces a novel Adaptive and Autonomous Schedule (AAS) for 6TiSCH networks, which dynamically adjusts transmit/receive (Tx/Rx) cells based on traffic demands and link PDR. The cell locations are determined using the MAC addresses of the sender and receiver nodes, along with their hop distances from the root. Additionally, the slotframe segment size is adapted according to the node count at a specific hop distance. This approach significantly reduces end-to-end latency, improves PDR, and extends network lifetime. The rest of the paper is organized as follows: Section 2 discusses the background, Section 3 explores the related work, Section 4 introduces the proposed scheduling algorithm, Section 5 focuses on the analytical model, performance evaluation is discussed in Section 6, and the paper is concluded in Section 7.

2. Background

The IPv6 over the TSCH mode of IEEE 802.15.4e (6TiSCH) protocol has been developed to integrate the IPv6 infrastructure with the underlying low-power IEEE 802.15.4e TSCH network [5,6]. It constructs and manages schedules based on TSCH network requirements, ensuring standardized communication between devices. For industrial Low-Power and Lossy Networks (LLNs) operating in a critical environment, it guarantees reliable, secured [7], and deterministic operations [8]. The 6TiSCH uses an existing IPv6 upper stack that includes an IPv6 over Low-Power Wireless Personal Area Networks (6LoWPAN) header compression layer, IPv6 Routing Protocol for LLNs (RPL), and Constrained Application Protocol (CoAP) lightweight application layer as shown in Figure 2.

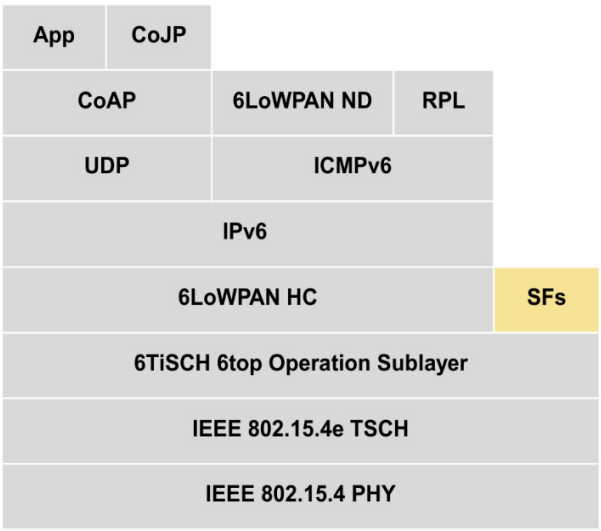

Figure 2. Layered architecture of the 6TiSCH protocol stack, integrating the IEEE 802.15.4e TSCH MAC layer with IPv6 protocols like 6LoWPAN, RPL, and CoAP.

The wide deployment in industrial LLNs consists of a set of routers that are interconnected with resource-constrained devices. To meet routing requirements in such network scenarios, the RPL routing protocol has been developed [9]. It organizes the network into a Destination Oriented Directed Acyclic Graph (DODAG) with several source nodes and a root node. The LLN may consist of multiple DODAGs created for a network and may run multiple instances of RPL concurrently. It builds a DODAG network route by keeping less signal overhead and specifies an objective function that enables the QoS requirements for specific applications. RPL isolates routing optimization from packet processing and forwarding operations. Routing optimization aims to achieve bounded latency while satisfying given constraints. In RPL, each node selects its parent node based on the optimized path to ensure root reachability [10].

6TiSCH defines the Operation Sublayer (6top), which enables distributed scheduling by negotiating the required number of cells between neighbors. It adds and/or removes cells within a slotframe as per traffic and topology needs. Additionally, it monitors the performance of each cell and relocates underperforming cells as needed. The 6P requests and responses are sent via Information Elements (IEs) that are encapsulated within the 6P packet. When a node sends an ADD REQUEST, it includes a list of required cells by specifying their [*timeOffset*, *channelOffset*] coordinates and the desired cell count. Upon receiving the ADD REQUEST, the neighbor node selects appropriate cells and replies with a RESPONSE containing the selected cell list. Cell negotiation is managed using either a 2-step or 3-step transaction. In the 2-step process, the sender directly selects cells for addition, deletion, or relocation. In the 3-step process, the receiver responds with a list of selected cells after evaluating the request. Any node can initiate a DELETE request to remove an allocated cell from the schedule.

Various 6TiSCH Scheduling Functions (SFs) are being developed to ensure deterministic operations in dynamic IIoT networks. These are broadly categorized based on their cell allocation approaches, which include centralized, distributed, and autonomous. The centralized schedulers rely on a single central unit to generate and manage the schedule for the entire network. This approach is well-suited for networks with fixed traffic patterns and low node density. However, the centralized mechanism becomes ineffective in dynamic network topologies with high node density and unstable data traffic [11]. The distributed scheduling approach utilizes cell negotiation to allocate cells between node pairs based on traffic and topology requirements. Both centralized and distributed schedulers experience signal overhead in dynamic networks. In contrast, the autonomous scheduling approach is becoming more attractive due to its low signal overhead and improved QoS [12]. It avoids both cell negotiation and centralized schedule management; instead, it uses the node's MAC addresses to allocate cells within a slotframe. Furthermore, a hybrid approach can be adopted for specific application requirements by combining multiple scheduling techniques.

In a dense network, heavy non-periodic traffic causes network congestion, leading to packet loss. In such scenarios, delivering packets to their destination within a deterministic time becomes more challenging. While retransmissions can address packet failures, they also increase end-to-end latency [13]. Consequently, an adaptive SF is becoming more appropriate for managing data packets based on their criticality level and efficiently allocating resources in multi-hop networks.

3. Related Work

Several 6TiSCH SFs have been proposed in the literature, employing centralized, distributed, and autonomous management approaches. While centralized schedulers are consistent, they introduce higher signal overhead due to the need for network information exchange and connectivity maintenance [14,15]. Consequently, they limit network scalability and are more suitable for static environments. On the other hand, distributed schedulers [16,17] are well-suited for dynamic networks, offering lower signal overhead, improved scalability, and enhanced reliability. A Distributed Broadcast-based Scheduling (DeBraS) algorithm [18] has been introduced for dense deployments. DeBraS reduces network traffic during control information exchange, minimizing collisions and improving throughput. Lowering collision rates reduces retransmissions, thereby decreasing latency and enhancing scalability in highly dense networks.

In LDSF [19] distributed scheduler, two neighbors negotiate dedicated cells for communication. It divides a slotframe into smaller blocks that repeat over time. It allocates one primary cell in a specified block and several ghost cells for retransmissions in consecutive

blocks. The transmitter uses ghost cells when it needs retransmission. This technique maintains the lower end-to-end delay by utilizing smaller block sizes. Distributed schedulers are appropriate for a dynamic network to some extent, as they involve cell negotiation signal overhead between pairs of nodes. DeBraS and LDSF suffer from higher energy consumption since cells are active more often to negotiate with neighbor nodes.

The Stripe is a distributed algorithm [20] that minimizes packet delay by placing cells chronologically. It considers node location in the RPL tree and sub-tree weight. The algorithm has two phases: relocation and reinforcement. Relocation moves cells to Stripe along the route, while downward traffic is combined into broadcast cells. The reinforcement phase allocates supplementary cells based on node traffic. Through optimized cell allocation, Stripe achieves faster convergence, minimal delay, and improved PDR. Wave [21] is a distributed scheduler that aims to minimize delays by limiting the cell count in the schedule. The algorithm schedules cells in consecutive waves, allocating them to nodes with packets for transmission. Subsequent waves are optimized versions of the initial wave. Cells are replicated in the next wave for nodes with multiple transmissions. Wave maintains a minimum number of cells, allowing packets to be forwarded from source to root in the same slotframe and ensuring all nodes have enough cells for their own and descendant packets.

The third approach, autonomous SF, is more suitable for highly dense networks. Several autonomous scheduling algorithms have been developed that do not comprise central management, information negotiation, signaling, or path resource reservation [22,23]. Nodes build and manage their schedule locally and autonomously by using their own and/or neighbor MAC addresses. Orchestra [24] is a node-based state-of-the-art autonomous scheduling scheme. It uses different slotframes for the TSCH beacon, RPL control messages, and application data. It uses one common shared slot at a fixed coordinate, [0, 1] of a slotframe for all nodes in the network to transmit and receive control messages. It defines different scheduling mechanisms: Receiver-Based Shared Orchestra (RBSO) and Sender-Based Shared Orchestra (SBSO). In RBSO, any node builds a single Rx slot by using its own MAC address, and Tx cells correspond to each neighbor by using the MAC addresses of neighbors. RBSO may result in a contention problem and packet drop as the node schedules a single Rx cell to receive packets from all the neighbors. In SBSO, a node uses its own MAC address to build a single Tx cell, and the sender's MAC address is used to build one Rx cell per neighbor. This approach offers fewer contentions compared to RBSO, as it employs an Rx cell per neighbor. However, it does result in higher energy consumption, as each Rx cell needs to wake up to sense the channel.

ALICE [25], a link-based autonomous scheduling scheme, has been developed, where authors claim that node-based schedulers are inefficient. Unlike Orchestra, this method does not suffer from the contention problem as it allocates a unique cell for every link. It uses a minimum slotframe size equal to $(2N - 2)/3$ for a network with N nodes and 2N-2 directional links. It determines the cell location by hashing the MAC addresses of the sender and receiver. The hash function can schedule the identical cell for multiple links. To address this problem, ALICE uses ASN to vary the cell location in every slotframe. Autonomous and traffic-aware scheduling [26], an enhanced Orchestra-based scheduler for TSCH networks, has been proposed that schedules cells dynamically based on traffic handled by the nodes. This technique significantly reduces delay while maintaining higher PDR. Monitoring the packets in a queue evaluates the cell count to be allocated for a node. Without any information negotiation, it computes the number of Tx cells towards its parent. The time offset of the cell is calculated using the Orchestra method, which allocates consecutive cells equal to pending packets at the node. Accordingly, the number of Rx cells for the parent node is adjusted. By allocating multiple cells as per traffic conditions, this

technique improves performance in terms of latency. However, this scheme becomes less scalable in dense networks, as consecutively allocated cells may collide.

The A^3 Adaptive Autonomous Allocation scheme [27], integrated with existing autonomous scheduling protocols such as Orchestra and ALICE, suggests a solution to address this problem. A^3 dynamically selects the slots in a slotframe to accommodate changing traffic needs. Its main advantage is the ability to estimate receiver traffic without requiring explicit control messages. It enhances throughput and PDR while reducing latency compared to previous methods. It allocates one primary cell and several secondary cells. The slotframe is divided into different zones, and one is used for a primary cell allocation while the other zones are for secondary cells, a maximum of up to eight cells. When the cell utilization exceeds the specified higher threshold, the number of secondary cells is doubled. Conversely, they are reduced by half when the utilization is below the specified lower threshold. Also, it uses an ASN number to change the cell location in every slotframe, which avoids collision on any particular slot. TESLA [28] is designed to optimize energy consumption and ensure reliable packet delivery in wireless networks. Nodes monitor the traffic load and adjust their Rx slot scheduling accordingly. When a node detects idle Rx slots, it expands its slotframe size to conserve energy by minimizing idle listening. Conversely, when facing a high packet flood, the node reduces its slotframe size to minimize contention and packet drop among neighboring nodes. TESLA enables dynamic adjustment of the slotframe size based on incoming traffic load without incurring additional control overhead.

While scheduling schemes like Orchestra, ALICE, and A^3 support autonomous and adaptive operations, they present significant limitations. Orchestra uses node-based periodic scheduling but fails to address traffic heterogeneity. Its fixed slotframes lead to over-provisioning of cells for sparse node distributions and under-provisioning in dense regions. This results in high contention near the root due to shared Rx slots when multiple child nodes attempt to send packets to the same parent. Moreover, Orchestra does not account for topology changes or traffic bursts, affecting both latency and PDR. ALICE improves latency by employing link-based scheduling and leveraging parent–child relationship knowledge through RPL topology. However, it still relies on static slot assignments, making it inefficient under dynamic traffic conditions. It also suffers from cell overlap and slot contention in dense deployments, particularly when multiple links compete for limited slotframe space. A^3 introduces traffic adaptivity using EWMA-based load estimation but lacks topology-aware slot segmentation. Its adaptation strategy ignores node depth and subtree size, leading to inefficiencies in multi-hop networks. These shortcomings highlight the need for a solution that integrates hop-based slot allocation, traffic prediction, and link quality awareness to enhance latency, delivery ratio, and energy efficiency in dynamic network environments.

Existing autonomous and adaptive scheduling approaches, in worst-case scenarios, may require multiple slotframes to achieve end-to-end packet delivery. Nodes with heavy traffic load result in queue overflow, higher latency, and contention [29–31]. A 6TiSCH low-latency autonomous scheduler [32] has been developed to schedule cells for all nodes at a given RPL level in a particular segment. A traffic-aware autonomous scheduler [33] assigns supplementary cells for the heavily loaded nodes. Though they improve performance compared to the existing Orchestra and ALICE schedulers, they lack adaptability to changing topology and traffic. Like Orchestra and ALICE, they also allocate single cells per link in every slotframe, leading to contention probability and packet drop. Ultimately, there is further scope for improvement in the scheduling techniques for reliable data transmission in time-constrained heavy-load networks. The algorithm proposed in this work addresses these challenges by adaptively allocating cells and segmenting slotframes based on traffic

demands while also incorporating link performance metrics. It creates a schedule based on the network topology and node density at a specified RPL level.

4. Proposed Adaptive Scheduling Technique

We propose a novel adaptive and autonomous scheduling technique for the network with non-periodic traffic patterns. Like ALICE, the proposed scheme utilizes node addresses to generate the schedule autonomously. The novelty of the proposed scheme lies in its dynamic allocation of Tx cells based on key network metrics such as queue backlog, link PDR, and hop count. Furthermore, the algorithm employs slotframe segmentation to ensure successive segment allocation for each link along the end-to-end communication path, thereby enhancing transmission efficiency and reliability. Moreover, by estimating the total number of transmissions required for nodes at a given hop distance, the scheme dynamically adjusts the slotframe segment size in accordance with the number of nodes and their respective hop distances. A data collection mesh network containing 60 nodes is considered for analysis, representing a typical scenario in IIoT applications. For simplicity, Figure 3 shows a partial network structure of 14 nodes. RPL converts the mesh topology into a tree structure, where the root node acts as the sink for data packets transmitted by its descendant nodes. In such a configuration, nodes situated closer to the root experience significantly higher traffic loads, as they are responsible for forwarding packets generated by their respective subtrees. The end-to-end latency in packet transmission is mainly affected by the number of slotframes required to forward a packet to its destination. The proposed technique utilizes the node's hop count to schedule cells within the slotframe, allocating a dedicated segment for all links at a specific hop distance in the RPL tree. This slotframe segmentation approach significantly reduces end-to-end latency by ensuring packet delivery within a single slotframe. Additionally, by minimizing contention probability, the technique enhances both the PDR and overall transmission reliability.

Given a maximum hop distance, H, in the network, the slotframe is partitioned into H segments of variable lengths, as illustrated in Figure 4. Cells for each link between nodes at a specific hop distance are allocated within the designated segment of the slotframe. The number of cells scheduled for a particular link is determined by the packet count in the sender's queue. Let $child(k)$ denote the set of child nodes of node k, and $P_{rec}(i)$ represent the number of packets received from a child node i. If the number of packets generated at sender node k is denoted as $P_{gen}(k)$, then the total number of packets P_k at node k is given by

$$P_k = P_{gen}(k) + \sum_{\forall i \in child(k)} P_{rec}(i) \tag{2}$$

Under ideal conditions with 100% link-layer PDR, a single transmission is sufficient for successful delivery. However, when the PDR is lower, retransmissions are required. If A_{kl} denotes the average number of transmissions required per packet over the link $(k \rightarrow l)$, which is the reciprocal of the average link-layer PDR, represented by PDR_{kl}. Accordingly, the total number of transmissions D_{Tk} over the link can be expressed as

$$D_{Tk} = A_{kl} \cdot P_k \tag{3}$$

Each node is assigned a specific segment for cell allocation. The total number of available cells within any segment is constrained by the segment length and the number of channel offsets. Although a segment in a longer slotframe offers more available cells, it can adversely impact end-to-end transmission latency. The pending packet count in the sender's queue determines the number of cells scheduled for a given link. However, in dense networks with high traffic, overprovisioning cells equal to the queue size may lead to cell collisions and increased energy consumption. Hence, to ensure the minimum collision

and packet drop with optimal slotframe length, the number of Tx cells B_{Tkl} allocated to node k is computed as the floor of the logarithm of the transmission traffic D_{Tk}.

$$B_{Tkl} = \lfloor log_2 D_{Tk} \rfloor + 1 \tag{4}$$

Node k allocates Tx cells using its own MAC address along with address of the receiver node l, and allocates Rx cells based on its own MAC address and those of its child nodes. The sender node k communicates the number of scheduled Tx cells to the receiver node l via a reserved field in the packet header. To minimize end-to-end latency, each node allocates Rx cells within one segment and Tx cells in the immediately following segment. If the hop count of the sender node k is h, the Tx cells are allocated in the $(H-h)^{th}$ segment of the slotframe.

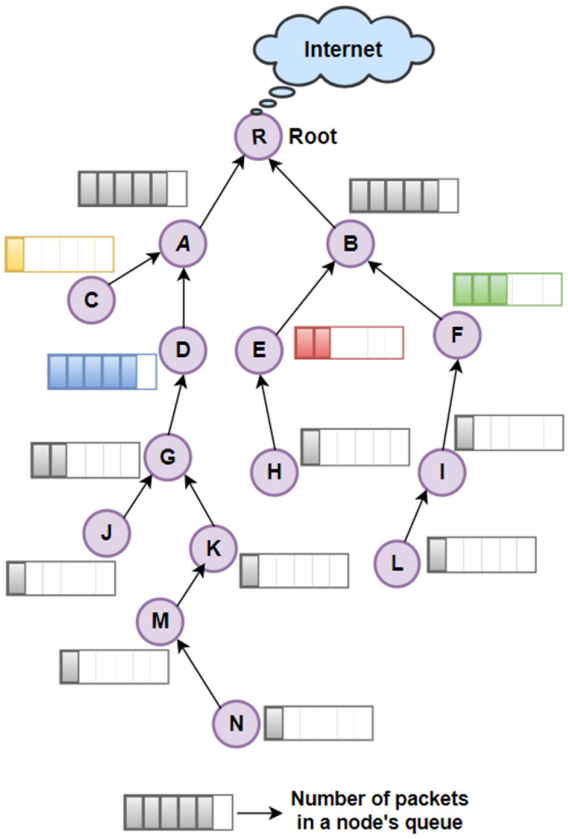

Figure 3. 6TiSCH data collection mesh network represented as an RPL tree with varying hop distances and traffic loads.

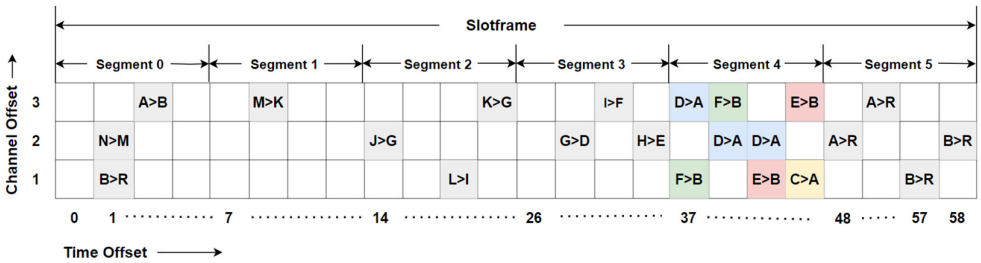

Figure 4. Proposed adaptive and autonomous scheduling slotframe structure using segmentation scheme for 6TiSCH networks.

Let n_h denote the node density at hop distance, h. The total number of cells, B_h, allocated for all links between the sender nodes, n_h, and their respective parent nodes, l, within the corresponding segment is given by

$$B_h = \sum_{\forall k \in n_h} B_{Tkl} \tag{5}$$

This approach adapts the number of scheduled cells based on the pending packets in a node's queue and the corresponding link-layer PDR. However, the number of allocated cells increases logarithmically rather than linearly, thereby preventing overprovisioning and reducing contention.

Furthermore, to minimize end-to-end latency and improve slotframe utilization, we propose the use of variable-length slotframe segments for links at different hop levels. By utilizing available channel offsets, C_{nu}, the minimum size of a slotframe segment is determined by ensuring sufficient allocation of Tx/Rx cells for nodes at hop distance, h, as given by

$$L_{hs_{min}} \geq \left\lceil \frac{n_h}{C_{nu}} + log_2(H - h + 1) \right\rceil \tag{6}$$

A larger slotframe segment is assigned to RPL levels with higher node density. Moreover, nodes located closer to the root typically handle greater traffic than other nodes, often resulting in buffer overflows. To mitigate this, segment sizes are increased for nodes nearer to the root. Taking into account the higher traffic at lower hop distances, the segment size is computed as the ceiling value derived from node density, the number of available channel offsets, and the logarithm of the hop distance. On the contrary, links at higher hop distances are assigned smaller segment lengths. The minimum slotframe length, SF_{min}, is determined as the sum of the lengths of all segments.

$$SF_{min} = \sum_{h=1}^{H} L_{hs_{min}} \tag{7}$$

The time offset for each cell is determined using a hash function applied to the addresses of the sender and receiver. According to (4), B_{Tkl} cells are scheduled for the link $(k \rightarrow l)$, where the time offset of the ith cell within a segment of size L_{hs} given by

$$to_{kl}(i) = \begin{cases} to + \sum_{d=(h+1)}^{H} L_{ds}, & h = 1 \; to \; H-1 \\ to\,, & h = H \end{cases} \tag{8}$$

where $to = hash[(k_{MAC} + i) + l_{MAC}]\%L_{hs}$ for $i = 0,\ 1, \ldots\ldots, (B_{Tkl} - 1)$, k_{MAC} and l_{MAC} are MAC addresses of node k and l, respectively.

The hash function used to determine the time offset ensures that cell allocation is uniformly distributed across the segment by combining sender and receiver MAC addresses along with segment length. This uniform distribution minimizes the collision among multiple nodes within the same segment. Additionally, since the hash function depends on unique MAC pairs, it avoids overlap across dissimilar links. The use of a modulo operator with the segment length ensures that cell allocation remains within the same segment.

As derived in (5), the proposed AAS strategy schedules B_h unique cells within the $(H - h)^{th}$ segment for all links connecting sender nodes at hop distance h to their respective parent nodes. To accommodate the high traffic generated by nodes one hop away from the root, segment 0 can be utilized additionally alongside the last segment. This strategy provides an extra opportunity for such nodes to forward packets without causing interference with pre-allocated transmissions. By using this segmentation approach, cells are scheduled in consecutive segments according to the node's hop distance from the root. The minimal-length slotframe segment is determined based on the number of pending

packets in the transmission queues and the number of active links at a given level of the RPL DODAG tree. In this work, the traffic load is predicted by using the current queue backlog at each node along with the hop distance traffic generation in the RPL topology. Since nodes closer to the root experience higher traffic due to forwarding packets of their descendants, our algorithm assumes traffic increases proportionally as the hop distance to the root decreases. The collective traffic arriving at a node, from its children and itself, is an effective predictor of upcoming load. This estimated load impacts the number of cells allocated, using the logarithmic mapping in Equation (4) to scale scheduling based on queue occupancy and links PDR.

The channel offset for the application data packet is determined by using the sender's MAC address and the total number of available channel offsets, as expressed by

$$co_{uc}(k) = hash[k_{MAC}]\%C_{nu} + 1 \tag{9}$$

The technique utilizes channel offsets 1, 2, and 3 for application data. The channel offset for the EB frame is fixed at 0, while for RPL it is set to 1. The time offset for the EB frame is determined based on the sender's MAC address, whereas the time offset for RPL remains constant at 0.

5. Parameter Analysis

A mathematical analysis is conducted for key performance parameters, including end-to-end latency, contention probability, and network lifetime, as detailed below. These metrics are crucial for evaluating network performance in IIoT applications operating under stringent environmental conditions.

5.1. End-to-End Latency

In a 6TiSCH network, end-to-end latency in transmission depends on the slotframe size and the PDR of the links along the communication path from the source to the root node. Links with lower PDR values require more retransmissions, thereby increasing the overall latency. The end-to-end latency EL_t in the proposed AAS approach, for a path of length P, is expressed as the summation of latencies introduced at each hop from the source node to the root and is determined as

$$EL_t = \sum_{h=1}^{P} \left(T_h + \frac{L_{hs} + E_{Txh} \cdot SF}{PDR_{kl}} \right) \tag{10}$$

where T_h denotes the number of time slots between the Rx cell from a child and the Tx cell to the parent for a node k at hop distance h. E_{Txh} represents the average number of retransmissions required at that hop, and PDR_{kl} is the PDR of the link between node k and its parent l. In scheduling schemes such as Orchestra, ALICE, and A^3, end-to-end latency is a function of the slotframe size. In contrast, the proposed AAS technique expresses latency solely as a function of the segment size L_{hs}. When retransmissions are required, the latency increases by an amount equivalent to the slotframe size.

5.2. Contention Probability

Contention may arise in the network when insufficient Tx/Rx cells are scheduled, leading to overlapping transmission attempts. The probability of contention in transmission depends on the network traffic load and the number of scheduled cells within a slotframe. Assuming a timeslot duration of 10 ms and that each node generates one packet every I_t ms the traffic T_r per slot can be expressed as $T_r = 10/I_t$.

The average contention probability in the proposed AAS scheme over a slot is given by

$$P_{cont} = 1 - e^{Tr} \tag{11}$$

From (5), the total number of cells allocated for all nodes within a slotframe is

$$B = \sum_{h=1}^{H} B_h \tag{12}$$

When the total available cells in a slotframe are given as $SF{\cdot}c_{nu}$, of these, B cells are scheduled for data transmission and reception, over which the network traffic is distributed. The contention probability, P_{cont}, can then be expressed as

$$P_{cont} = 1 - e^{-\frac{Tr \cdot SF \cdot C_{nu}}{B}} \tag{13}$$

The network experiences higher contention when nodes handle heavy traffic, especially when the slotframe has a sparse distribution of cells. In the proposed technique, by adapting the optimal number of cells, the contention probability is reduced by a factor of B. This results in improved PDR and enhanced transmission reliability.

5.3. Network Lifetime

In a 6TiSCH network, the network lifetime is determined by the minimum lifetime of the nodes closest to the root. If the battery of such nodes is depleted, network connectivity is disrupted. A node's lifetime is primarily influenced by its average energy consumption during each slotframe. In smaller slotframes, timeslots repeat more frequently, enabling higher transmission rates. However, this results in nodes remaining active more often, leading to increased energy consumption. Conversely, a larger slotframe size can cause higher latency and reduced PDR, as nodes have fewer transmission opportunities. Therefore, the slotframe size can be selected based on the critical performance parameters specific to the application's requirements.

An energy consumption model based on real measurement has been used for the analysis [19]. It describes the energy consumed by every node in each type of cell. Taking the node's initial energy of $10,157.4 \times 10^6$ µc, average consumption is determined as the sum of the energy spent in every slot. A node in IEEE 802.15.4e TSCH consumes energy in 6 different types of time slots: 54.5 µc in TxDataRxAck to send application data and receive an acknowledgment (ACK); 32.6 µc in RxDataTxAck to receive data and sends back an ACK, 49.5 µc in TxData to sends broadcast packet without an ACK; 22.6 μc in RxData to receive without ACK; 6.4 µc in Idle slot during which a node listens for data but does not receive any; and 0 µc in a sleep slot, where the node's radio remains turned off. Node's lifetime $life(n)$ in a year is given as

$$life(n) = \frac{E(n){\cdot}SF}{E_c(n){\cdot}3600{\cdot}24{\cdot}365} \tag{14}$$

where $E(n)$ is the initial energy of a node n in µc, and $E_c(n)$ is the average energy consumed by the node during a slotframe in µc. The minimum lifetime of any node, expressed in years, determines the network lifetime and is given by

$$Net_{life} = \min_{n \in N} life(n) \tag{15}$$

In energy-critical applications, longer network lifetime can be achieved by employing longer slotframes.

6. Performance Evaluation

The proposed AAS is implemented in the Contiki-NG operating system and evaluated using the Cooja simulator, which is a widely used tool for IIoT simulations [34,35]. A network of 60 nodes arranged in an 8×5 (800 m $\times$ 500 m) grid is modeled, with one node designated as the root and the remaining as sources. A typical IIoT data collection scenario is considered, wherein the root node serves as the sink for all packets transmitted by the sources, with a maximum hop distance of 6. The packet transmission and interference ranges are both set to 100 m. The simulation is carried out with varying packet intervals from 2 s to 15 s and slotframe size from 31 slots to 101 slots, representing realistic IIoT data collection patterns. To evaluate the proposed technique, key performance parameters such as average end-to-end latency, contention probability, PDR, and network lifetime were analyzed by varying the slotframe size and packet generation intervals.

As end-to-end latency tends to increase with slotframe size, smaller slotframes are preferable for time-critical applications. However, smaller slotframes lead to higher energy consumption, as nodes remain active more frequently within a shorter time span. Consequently, for energy-constrained applications, larger slotframes are more suitable, though at the cost of increased latency. Additionally, larger slotframes may lead to a reduction in PDR due to an increased contention probability. With fewer transmission opportunities available in such slotframes, packet queues tend to grow longer, thereby increasing the risk of packet drops.

The proposed scheduler minimizes end-to-end latency by ensuring the delivery of packets from source to destination within a single slotframe. An additional slotframe is required only in cases of retransmission. Cells are allocated in successive segments along the route from source to destination, with segment lengths dynamically adapted based on the number of nodes at each level and the traffic they handle. The effectiveness of the proposed AAS has been evaluated and compared with state-of-the-art scheduling approaches, including the node-based autonomous technique Orchestra, the link-based ALICE scheduler, and A^3 integrated with ALICE, as illustrated in Figure 5.

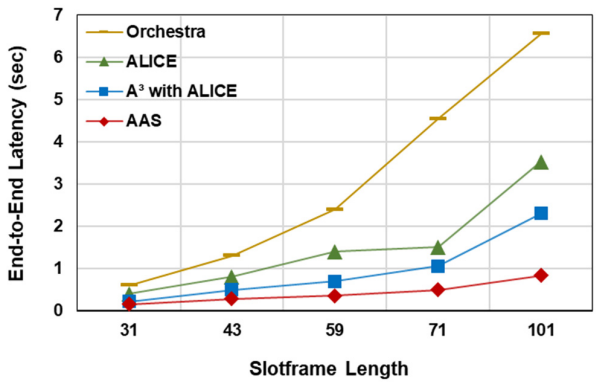

Figure 5. End-to-end latency versus slotframe length.

Unlike these schedulers, AAS models latency as a function of segment length rather than slotframe length. The number of nodes determines the segment length at a given rank and the corresponding link PDR. Orchestra allocates a unique cell to each node within a slotframe, using the sender's or receiver's address. Similarly, ALICE assigns a unique cell to each link in the network. These schemes incur higher latency, as multiple slotframes are required for end-to-end packet delivery. In the worst-case scenario, a cell with time offset to_{kl} may be allocated for the link $(k \rightarrow l)$, while $to_{kl} + 1$ is allocated for the link $(j \rightarrow k)$. This hop-to-hop scheduling leads to a maximum delay of $slotframe_{length} - 1$ per hop.

Consequently, the worst-case end-to-end latency over a path with hops, h, can approach h slotframes.

A^3 partitions the slotframe into multiple zones based on the traffic load at each node. Initially, it assigns primary cells to every link using the ALICE allocation strategy. When the utilization of primary cells exceeds a predefined threshold, secondary cells are allocated to accommodate the increased traffic. However, the limitation of A^3 is that it allocates primary cells anywhere within a slotframe. When the schedule relies solely on primary cells, the resulting latency is comparable to that of Orchestra. The inclusion of secondary cells in A^3 helps reduce latency by providing nodes with additional opportunities to forward packets. As shown in Figure 5, the proposed AAS technique outperforms existing approaches across all slotframe sizes. At a slotframe size of 31, the end-to-end latency achieved by AAS is 32% lower than A^3, 63% lower than ALICE, and 75% lower than Orchestra. For a slotframe with 101 timeslots, AAS exhibits 53% lower latency than A^3 and 67% lower than ALICE. Overall, AAS consistently achieves the lowest end-to-end latency among the compared techniques.

Contention probability is evaluated across varying slotframe sizes and packet intervals. The traffic load on each timeslot was estimated based on the traffic generated by the node and the slot duration, as described in (11). In larger slotframes with sparser cell distributions, high network traffic leads to increased contention due to the limited availability of cells per unit time. Figure 6 presents the contention probability for different scheduling techniques across multiple slotframe sizes, with a packet interval of 1 s. It is observed to be lower in AAS compared to other schedulers due to its adaptive cell allocation strategy, which enables more efficient resource utilization. At a slotframe size of 101, the contention probability in AAS is 30% lower than A^3, 58% lower than ALICE, and 71% lower than Orchestra. In A^3, contention is comparatively more due to the dispersion of primary and secondary cells across the slotframe. In Orchestra and ALICE, contention remains high as they schedule a single Tx cell in a slotframe for every node. Figure 7 shows the reduction in contention in AAS with increasing packet intervals. At a slotframe length of 31 timeslots, the contention is 72% lower when the packet interval is 5 s compared to an interval of 0.5 s.

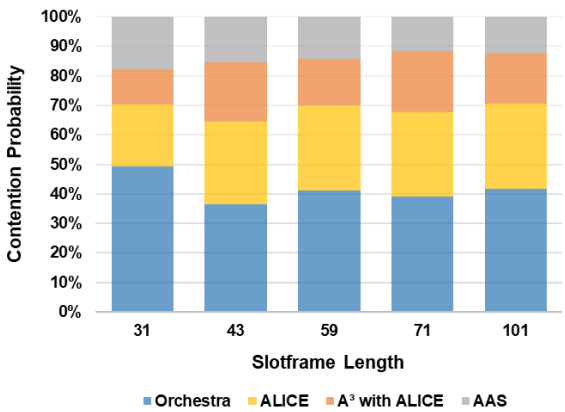

Figure 6. Contention probability versus slotframe length.

The average end-to-end PDR of the network is the ratio of the total number of packets received by the sink to the total number of packets transmitted by the sender. PDR in a network is affected with the increasing slotframe size, as a lesser number of cells are available for transmission. The results in Figure 8 show the end-to-end PDR of AAS for a slotframe size of 59 timeslots, and packet intervals are varied from 2 s to 15 s. The PDR of the AAS is 27% higher than A^3 when the packet interval is set to 2 s; however, it is 14% higher than Orchestra and 11% higher than ALICE when an interval is set to 10 s. PDR in Orchestra and ALICE is reduced due to the limited transmission opportunities within a

slotframe. In the case of A^3, transmission opportunity is increased by using multiple zones; however, for higher traffic conditions, PDR is dropped below AAS due to higher congestion probability. AAS assigns the cells within a specified segment as per traffic requirement and links PDR, resulting in a lower congestion problem.

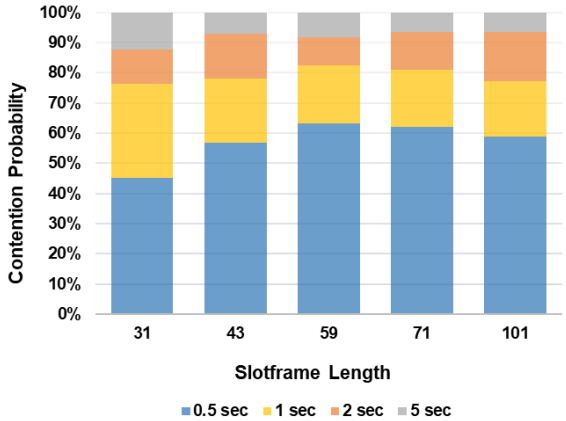

Figure 7. Contention probability versus slotframe length for different packet intervals.

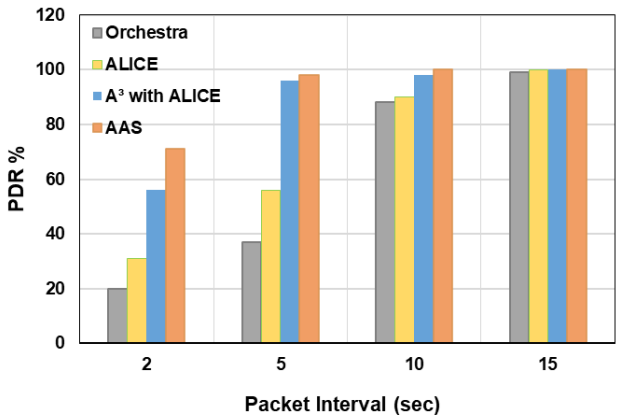

Figure 8. PDR versus packet interval.

The life of the network is decided by the node with the highest energy consumption. Nodes consume energy during active slots, used for data transmission, reception, and acknowledgment. The entire network may go down if any node, specifically closer to the root, runs out of energy. Nodes in Orchestra consume energy in idle listening at most of the time, as every node has a single Tx cell and Rx cells per neighbor. However, in A^3, the allocated cells are doubled when its utilization is above a threshold value, which leads to more energy consumption. The proposed AAS algorithm improves energy efficiency by adapting to traffic conditions, hop count, and link reliability. It avoids over-provisioning by allocating a logarithmic number of cells based on the estimated queue backlog, which significantly reduces idle listening and channel contention. Similarly, the slotframe segmentation based on hop distance ensures that nodes closer to the root, typically experiencing higher traffic accumulation, are allocated more communication resources, while other nodes save energy by operating with fewer active slots. Moreover, by incorporating link quality into the slot allocation strategy, AAS minimizes retransmissions due to poor links, further reducing unnecessary energy consumption. These mechanisms reduce radio-on time and improve duty-cycle performance, thereby extending the overall network lifetime. Experimental results demonstrate that AAS achieves lower energy consumption compared to other schemes like ALICE and A^3.

Figure 9 illustrates the network lifetime for different slotframe lengths. The result shows that when a packet interval is set to 1 min and the slotframe of 101 timeslots, the network lifetime in AAS is 10% higher than A^3, 23% higher than ALICE, and 64% higher than Orchestra. Node's energy consumption is higher for a shorter slotframe, as nodes wake up more frequently. It can be minimized by increasing the slot frame size. However, it is at the cost of higher end-to-end latency. For a smaller slotframe with higher traffic conditions, AAS consumes more energy, as it has more active cells for a short period than in the Orchestra. For simulation, we considered 60 nodes that represent a mid-scale industrial scenario, balancing complexity and real-world density. To evaluate the scalability of the proposed AAS mechanism, we conducted simulations using 40 node count, arranged in grid topologies of 6×5 (600 m $\times$ 500 m). The performance in terms of latency, PDR, and contention remains comparable, with AAS demonstrating its advantage over existing schedulers.

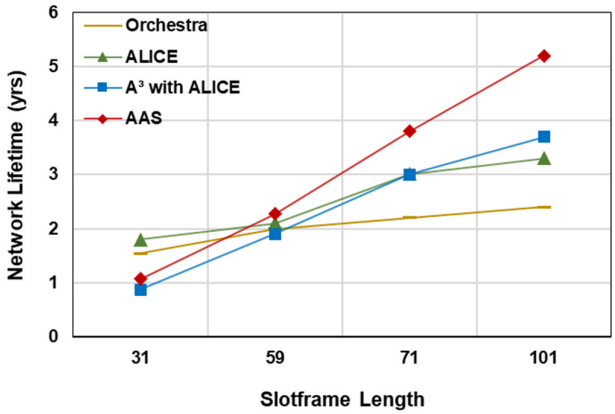

Figure 9. Network lifetime versus slotframe length.

7. Conclusions

This paper proposes a 6TiSCH adaptive slotframe allocation technique AAS for IIoT applications. This technique uses the node's MAC addresses and hops distance from the root to schedule the cell within a slotframe. The number of cells to be scheduled is estimated by monitoring the pending packets in a queue and the PDR of the link. By scheduling optimal cells, AAS improves the end-to-end latency and network lifetime. Mathematical analysis shows the performance of the AAS technique over current schedulers. Results show that the end-to-end latency of the AAS is up to 63% lower than the ALICE and 32% lower than the A^3. The PDR of AAS is 27% higher than A^3 for a packet interval of 2 s and 11% higher than ALICE for a packet interval of 10 s. The network lifetime with the AAS is up to 23% longer than ALICE and 10% longer than the A^3 scheduler. The results demonstrate that the proposed technique outperforms the existing approaches in terms of end-to-end latency, PDR, and network lifetime. This technique will significantly improve IIoT network performance.

Author Contributions: Conceptualization, N.P., B.S.C. and P.D.K.; methodology, N.P., B.S.C. and P.D.K.; software, N.P., B.S.C. and P.D.K.; validation, N.P., B.S.C. and P.D.K.; formal analysis, N.P., B.S.C. and P.D.K.; investigation, N.P., B.S.C. and P.D.K.; resources, N.P., B.S.C. and P.D.K.; writing—original draft preparation, N.P.; writing—review and editing, N.P., B.S.C. and P.D.K.; visualization, N.P., B.S.C. and P.D.K.; supervision B.S.C. and P.D.K. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: No new data were created or analyzed in this study. Data sharing is not applicable to this article.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Rose, K.; Eldridge, S.; Chapin, L. The Internet of Things (IoT): An Overview. *Int. J. Eng. Res. Appl.* **2015**, *5*, 71–82.
2. De Guglielmo, D.; Anastasi, G.; Seghetti, A. From IEEE 802.15.4 to IEEE 802.15.4e: A step towards the Internet of Things. *Adv. Intell. Syst. Comput.* **2014**, *260*, 135–152. [CrossRef]
3. Askhedkar, A.R.; Chaudhari, B.S. Energy Efficient LoRa Transmission over TV White Spaces. *Int. J. Inf. Technol.* **2023**, *15*, 4337–4347. [CrossRef]
4. Serror, M.; Hack, S.; Henze, M.; Schuba, M.; Wehrle, K. Challenges and Opportunities in Securing the Industrial Internet of Things. *IEEE Trans. Ind. Inf.* **2021**, *17*, 2985–2996. [CrossRef]
5. Thubert, P. An Architecture for IPv6 Over the Time-Slotted Channel Hopping Mode of IEEE 802.15.4 (6TiSCH). IETF RFC 9030, 2021. Available online: https://www.rfc-editor.org/info/rfc9030 (accessed on 8 June 2023).
6. Vilajosana, X.; Watteyne, T.; Chang, T.; Vucinic, M.; Duquennoy, S.; Thubert, P. IETF 6TiSCH: A Tutorial. *IEEE Commun. Surv. Tutor.* **2020**, *22*, 595–615. [CrossRef]
7. Gupta, S.; Chaudhari, B.S.; Chakrabarty, B. Vulnerable network analysis using war driving and security intelligence. In Proceedings of the 2016 International Conference on Inventive Computation Technologies (ICICT), Coimbatore, India, 26–27 August 2016; pp. 1–5. [CrossRef]
8. Rao, A.; Chaudhari, B.S. Development of LoRaWAN based Traffic Clearance System for Emergency Vehicles. In Proceedings of the 2020 Fourth International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC), Palladam, India, 7–9 October 2020; pp. 217–221. [CrossRef]
9. Winter, T.; Pascal, T.; Brandt, A.; Jonathan, H.; Richard, K.; Philip, L.; Kris, P.; Rene, S.; Jean-Philippe, V.; Roger, A. RPL: IPv6 Routing Protocol for Low Power and Lossy Networks. IETF RFC 6550 2012. Available online: https://datatracker.ietf.org/doc/html/rfc6550 (accessed on 12 June 2023).
10. Pister, K.; Phinney, T.; Thubert, P.; Dwars, S. Industrial Routing Requirements in Low-Power and Lossy Networks. IETF RFC 5673. 2009. Available online: https://www.rfc-editor.org/rfc/rfc5673 (accessed on 12 November 2023).
11. Askhedkar, A.; Chaudhari, B.; Zennaro, M.; Pietrosemoli, E. TV white spaces for low-power wide-area networks. In *LPWAN Technologies for IoT and M2M Applications*; Elsevier: Amsterdam, Netherlands, 2020; pp. 167–179. [CrossRef]
12. Vucinic, M.; Chang, T.; Skrbic, B.; Kocan, E.; Pejanovic-Djurisic, M.; Watteyne, T. Key performance indicators of the reference 6TiSCH implementation in internet-of-things scenarios. *IEEE Access* **2020**, *8*, 79147–79157. [CrossRef]
13. Askhedkar, A.; Chaudhari, B.; Zennaro, M. Hardware and software platforms for low-power wide-area networks. In *LPWAN Technologies for IoT and M2M Applications*; Elsevier: Amsterdam, Netherlands, 2020; pp. 397–407. [CrossRef]
14. Jin, Y.; Kulkarni, P.; Wilcox, J.; Sooriyabandara, M. A centralized scheduling algorithm for IEEE 802.15.4e TSCH based industrial low power wireless networks. In Proceedings of the IEEE Wireless Communications and Networking Conference, WCNC, Doha, Qatar, 3–6 April 2016. [CrossRef]
15. Palattella, M.R.; Accettura, N.; Dohler, M.; Grieco, L.A.; Boggia, G. Traffic aware scheduling algorithm for reliable low-power multi-hop IEEE 802.15.4e networks. In Proceedings of the IEEE International Symposium on Personal, Indoor and Mobile Radio Communications, PIMRC, Sydney, Australia, 9–12 September 2012. [CrossRef]
16. Daneels, G.; Spinnewyn, B.; Latré, S.; Famaey, J. ReSF: Recurrent Low-Latency Scheduling in IEEE 802.15.4e TSCH networks. *Ad. Hoc Netw.* **2018**, *69*, 100–114. [CrossRef]
17. Morell, A.; Vilajosana, X.; Vicario, J.L.; Watteyne, T. Label switching over IEEE802.15.4e networks. *Trans. Emerg. Telecommun. Technol.* **2013**, *24*, 458–475. [CrossRef]
18. Municio, E.; Latré, S. Decentralized broadcast-based scheduling for dense multi-hop TSCH networks. In Proceedings of the Annual International Conference on Mobile Computing and Networking, MOBICOM, New York, NY, USA, 3–7 October 2016. [CrossRef]
19. Kotsiou, V.; Papadopoulos, G.Z.; Chatzimisios, P.; Theoleyre, F. LDSF: Low-Latency Distributed Scheduling Function for Industrial Internet of Things. *IEEE Internet Things J.* **2020**, *7*, 8688–8699. [CrossRef]
20. Juc, I.; Alphand, O.; Guizzetti, R.; Favre, M.; Duda, A. *Stripe: A Distributed Scheduling Protocol for 802.15.4e TSCH Networks*; [Research Report] RR-LIG-54; Laboratoire d'Informatique de Grenoble: Saint-Martin-d'Hères, France, 2017; p. 6. Available online: https://hal.inria.fr/hal-01585904 (accessed on 8 June 2023).
21. Soua, R.; Minet, P.; Livolant, E. Wave: A distributed scheduling algorithm for convergecast in IEEE 802.15.4e TSCH networks. *Trans. Emerg. Telecommun. Technol.* **2016**, *27*, 557–575. [CrossRef]

22. Elsts, A.; Kim, S.; Kim, H.-S.; Kim, C. An empirical survey of autonomous scheduling methods for TSCH. *IEEE Access* **2020**, *8*, 67147–67165. [CrossRef]
23. Elsas, R.; Van Leemput, D.; Hoebeke, J.; De Poorter, E. 3MSF: A Multi-Modal Adaptation of the 6TiSCH Minimal Scheduling Function for the Industrial IoT. *Sensors* **2024**, *24*, 2414. [CrossRef]
24. Duquennoy, S.; Al Nahas, B.; Landsiedel, O.; Watteyne, T. Orchestra: Robust mesh networks through autonomously scheduled TSCH. In Proceedings of the 13th ACM Conference on Embedded Networked Sensor Systems, SenSys 2015, Seoul, Republic of Korea, 1–4 November 2015; pp. 337–350. [CrossRef]
25. Kim, S.; Kim, H.-S.; Kim, C. Alice: Autonomous link-based cell scheduling for TSCH. In Proceedings of the 2019 Information Processing in Sensor Networks, IPSN 2019, Montreal, QC, Canada, 16–18 April 2019; pp. 121–132. [CrossRef]
26. Rekik, S.; Baccour, N.; Jmaiel, M.; Drira, K.; Grieco, L.A. Autonomous and Traffic-aware Scheduling for TSCH Networks. *Comput. Netw.* **2018**, *135*, 201–212. [CrossRef]
27. Kim, S.; Kim, H.-S.; Kim, C. A^3: Adaptive Autonomous Allocation of TSCH Slots. in IPSN' 21. In Proceedings of the 20th International Conference on Information Processing in Sensor Networks, Nashville, TN, USA, 18–21 May 2021.
28. Jeong, S.; Paek, J.; Kim, H.-S.; Bahk, S. TESLA: Traffic-Aware Elastic Slotframe Adjustment in TSCH Networks. *IEEE Access* **2019**, *7*, 130468–130483. [CrossRef]
29. Chang, T.; Vučinić, M.; Guillén, X.V.; Dujovne, D.; Watteyne, T. 6TiSCH minimal scheduling function: Performance evaluation. *Internet Technol. Lett.* **2020**, *3*, e170. [CrossRef]
30. Hauweele, D.; Koutsiamanis, R.-A.; Quoitin, B.; Papadopoulos, G.Z. Thorough performance evaluation & analysis of the 6TiSCH minimal scheduling function (MSF). *J. Signal Process. Syst.* **2022**, *94*, 3–25. [CrossRef]
31. Righetti, F.; Vallati, C.; Gavioli, A.; Anastasi, G. Performance Evaluation of Adaptive Autonomous Scheduling Functions for 6TiSCH Networks. *IEEE Access* **2021**, *9*, 127576–127594. [CrossRef]
32. Pradhan, N.M.; Chaudhari, B.S.; Zennaro, M. 6TiSCH Low Latency Autonomous Scheduling for Industrial Internet of Things. *IEEE Access* **2022**, *10*, 71566–71575. [CrossRef]
33. Pradhan, N.; Chaudhari, B.S. Chaudhari. Traffic-aware autonomous scheduling for 6TiSCH networks. *Int. J. Comput. Appl.* **2022**, *44*, 1039–1046. [CrossRef]
34. Kurniawan, A. *Practical Contiki-NG*; Apress: New York, NY, USA, 2018. [CrossRef]
35. Duquennoy, S.; Elsts, A.; Al Nahas, B.; Oikonomo, G. TSCH and 6TiSCH for contiki: Challenges, design and evaluation. In Proceedings of the 2017 13th International Conference on Distributed Computing in Sensor Systems, DCOSS 2017, Ottawa, ON, Canada, 5–7 June 2017; pp. 11–18. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.